C - Nested If Statements

It is always legal in C programming to nest **if-else** statements, which means you can use one **if** or **else-if** statement inside another **if** or **else-if** statement(s).

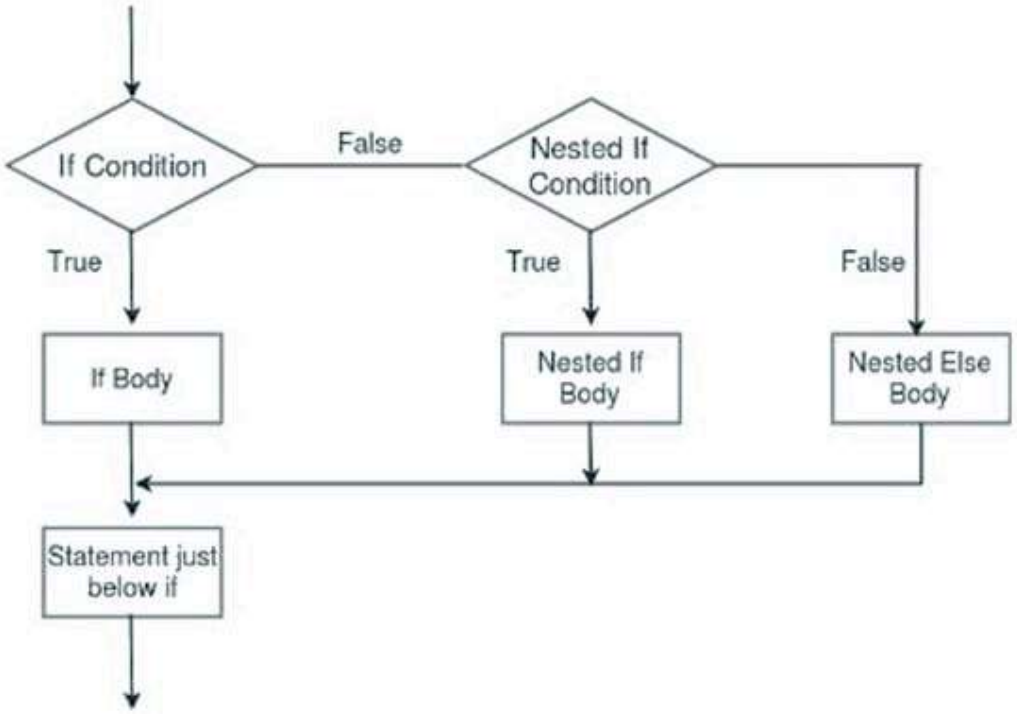
In the programming context, the term "nesting" refers to enclosing a particular programming element inside another similar element. For example, nested loops, nested structures, nested conditional statements, etc. If an **if** statement in C is employed inside another **if** statement, then we call it as a **nested if statement** in C.

Syntax

The syntax of **nested if** statements is as follows −

```
if (expr1){
   if (expr2){
      block to be executed when
      expr1 and expr2 are true
   }
   else{
      block to be executed when
      expr1 is true but expr2 is false
   }
}
```

The following flowchart represents the nesting of **if** statements −



You can compound the Boolean expressions with **&&** or **||** to get the same effect. However, for more complex algorithms, where there are different combinations of multiple Boolean expressions, it becomes difficult to form the compound conditions. Instead, it is recommended to use nested structures.

Another **if** statement can appear inside a top-level **if** block, or its **else** block, or inside both.

Example 1

Let us take an example, where the program needs to determine if a given number is less than 100, between 100 to 200, or above 200. We can express this logic with the following compound Boolean expression –

</> Open Compiler

```
#include <stdio.h>

int main (){

   // local variable definition
   int a = 274;
   printf("Value of a is : %d\n", a);

   if (a < 100){
      printf("Value of a is less than 100\n");
```

```
   }

   if (a >= 100 && a < 200){
      printf("Value of a is between 100 and 200\n" );
   }

   if (a >= 200){
      printf("Value of a is more than 200\n" );
   }
}
```

Output

Run the code and check its output. Here, we have intialized the value of "a" as 274. Change this value and run the code again. If the supplied value is less than 100, then you will get a different output. Similarly, the output will change again if the supplied number is in between 100 and 200.

```
Value of a is : 274
Value of a is more than 200
```


Example 2

Now let's use nested conditions for the same problem. It will make the solution more understandable when we use nested conditions.

First, check if "a >= 100". Inside the true part of the **if** statement, check if it is <200 to decide if the number lies between 100-200, or it is >200. If the first condition (a >= 100) is false, it indicates that the number is less than 100.

</> Open Compiler

```
#include <stdio.h>

int main (){

   // local variable definition
   // check with different values 120, 250 and 74
   int a = 120;
```

```
   printf("value of a is : %d\n", a );

   // check the boolean condition
   if(a >= 100){

      // this will check if a is between 100-200
      if(a < 200){
         // if the condition is true, then print the following
         printf("Value of a is between 100 and 200\n" );
      }
      else{
          printf("Value of a is more than 200\n");
      }
   }
   else{
       // executed if a < 100
       printf("Value of a is less than 100\n");
   }

   return 0;
}
```

Output

Run the code and check its output. You will get different outputs for different input values of "a" –

```
Value of a is : 120
Value of a is between 100 and 200
```

Example 3

The following program uses nested **if** statements to determine if a number is divisible by 2 and 3, divisible by 2 but not 3, divisible by 3 but not 2, and not divisible by both 2 and 3.

</> Open Compiler

```
#include <stdio.h>

int main(){
   int a = 15;
```

```
   printf("a: %d\n", a);

   if (a % 2 == 0) {
      if (a % 3 == 0){
         printf("Divisible by 2 and 3");
      }
      else {
         printf("Divisible by 2 but not 3");
      }
   }
   else {
      if (a % 3 == 0){
         printf("Divisible by 3 but not 2");
      }
      else{
         printf("Not divisible by 2, not divisible by 3");
      }
   }

   return 0;
}
```

Output

Run the code and check its output –

```
a: 15
Divisible by 3 but not 2
```

For different values of "a", you will get different outputs.

Example 4

Given below is a C program to check if a given year is a leap year or not. Whether the year is a leap year or not is determined by the following rules –

- Is the year divisible by 4?
- If yes, is it a century year (divisible by 100)?
- If yes, is it divisible by 400? If yes, it is a leap year, otherwise not.
- If it is divisible by 4 and not a century year, it is a leap year.

- If it is not divisible by 4, it is not a leap year.

Here is the C code –

</> Open Compiler

```
#include <stdio.h>

int main(){

   // Test the program with the values 1900, 2023, 2000, 2012
   int year = 1900;
   printf("year: %d\n", year);

   // is divisible by 4?
   if (year % 4 == 0){

      // is divisible by 100?
      if (year % 100 == 0){

         // is divisible by 400?
         if(year % 400 == 0){
            printf("%d is a Leap Year\n", year);
         }
         else{
            printf("%d is not a Leap Year\n", year);
         }
      }
      else{
         printf("%d is not a Leap Year\n", year);
      }
   }
   else{
      printf("%d is a Leap Year\n", year);
   }
}
```

Output

Run the code and check its output –

```
year: 1900
1900 is not a Leap Year
```

Test the program with different values for the variable "year" such as 1900, 2023, 2000, 2012.

The same result can be achieved by using the compound Boolean expressions instead of nested if statements, as shown below –

```
If (year % 4 == 0 && (year % 400 == 0 || year % 100 != 0)){
   printf("%d is a leap year", year);
}
else{
   printf("%d is not a leap year", year);
}
```

With **nested if** statements in C, we can write structured and multi-level decision-making algorithms. They simplify coding the complex discriminatory logical situations. Nesting too makes the program more readable, and easy to understand.

Switch Statement in C

A **switch statement** allows a variable to be tested for equality against a list of values. Each value is called a **case**, and the variable being switched on is checked for each switch case.

C switch-case Statement

The **switch-case** statement is a decision-making statement in C. The **if-else** statement provides two alternative actions to be performed, whereas the **switch-case** construct is a multi-way branching statement. A **switch** statement in C simplifies multi-way choices by evaluating a single variable against multiple values, executing specific code based on the match. It allows a variable to be tested for equality against a list of values.

Syntax of switch-case Statement

The flow of the program can switch the line execution to a branch that satisfies a given case. The schematic representation of the usage of switch-case construct is as follows −

```
switch (Expression){

   // if expr equals Value1
   case Value1:
      Statement1;
      Statement2;
      break;

   // if expr equals Value2
   case Value2:
      Statement1;
      Statement2;
      break;
      .
      .
   // if expr is other than the specific values above
   default:
      Statement1;
      Statement2;
}
```

How switch-case Statement Work?

The parenthesis in front of the **switch** keyword holds an expression. The expression should evaluate to an integer or a character. Inside the curly brackets after the parenthesis, different possible values of the expression form the case labels.

One or more statements after a colon(:) in front of the case label forms a block to be executed when the expression equals the value of the label.

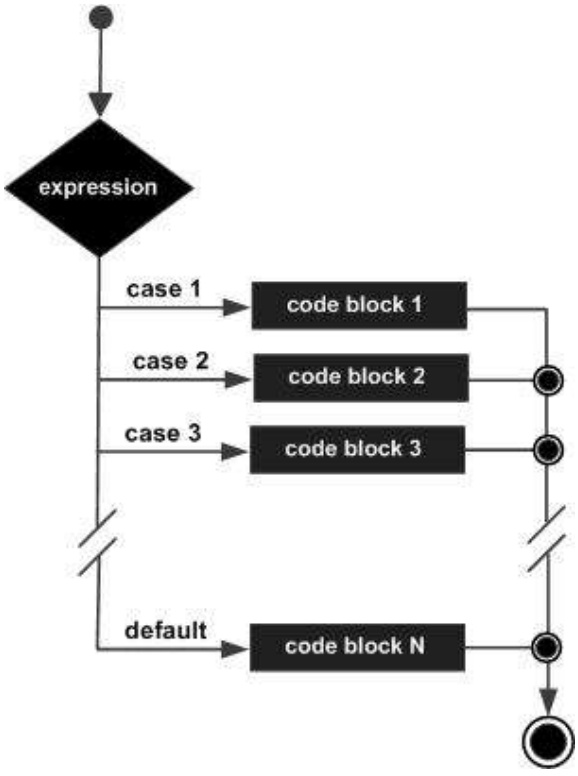
You can literally translate a switch-case as "in case the expression equals value1, execute the block1", and so on.

C checks the expression with each label value, and executes the block in front of the first match. Each case block has a **break** as the last statement. The break statement takes the control out of the scope of the switch construct.

You can also define a **default case** as the last option in the switch construct. The default case block is executed when the expression doesn't match with any of the earlier case values.

Flowchart of switch-case Statement

The flowchart that represents the switch-case construct in C is as follows –



Rules for Using the switch-case Statement

The following rules apply to a switch statement –

- The expression used in a **switch** statement must have an integral or enumerated type, or be of a class type in which the class has a single conversion function to an integral or enumerated type.
- You can have any number of **case** statements within a switch. Each **case** is followed by the value to be compared to and a colon.
- The constant-expression for a case must be the same data type as the variable in the switch, and it must be a constant or a literal.
- When the variable being switched on is equal to a case, the statements following that case will execute until a break statement is reached.
- When a break statement is reached, the switch terminates, and the flow of control jumps to the next line following the switch statement.
- Not every case needs to contain a break. If no break appears, the flow of control will fall through to subsequent cases until a break is reached.
- A switch statement can have an optional default case, which must appear at the end of the switch. The default case can be used for performing a task when none of the cases is true. No break is needed in the default case.

The **switch-case statement** acts as a compact alternative to cascade of if-else statements, particularly when the Boolean expression in "if" is based on the "=" operator.

If there are more than one statements in an **if** (or **else**) block, you must put them inside curly brackets. Hence, if your code has many **if** and **else** statements, the code with that many opening and closing curly brackets appears clumsy. The switch-case alternative is a compact and clutter-free solution.

C switch-case Statement Examples

Practice the following examples to learn the switch case statements in C programming language –

Example 1

In the following code, a series of if-else statements print three different greeting messages based on the value of a "ch" variable ("m", "a" or "e" for morning, afternoon or evening).

</> Open Compiler

```
#include <stdio.h>

int main(){

   /* local variable definition */
```

```
   char ch = 'e';
   printf("Time code: %c\n\n", ch);

   if (ch == 'm')
      printf("Good Morning");

   else if (ch == 'a')
      printf("Good Afternoon");
   else
      printf("Good Evening");

   return 0;
}
```

The if-else logic in the above code is replaced by the switch-case construct in the code below –

</> Open Compiler

```
#include <stdio.h>

int main (){

   // local variable definition
   char ch = 'm';
   printf("Time code: %c\n\n", ch);

   switch (ch){

      case 'a':
         printf("Good Afternoon\n");
         break;

      case 'e':
         printf("Good Evening\n");
         break;

      case 'm':
         printf("Good Morning\n");
   }
```

```
   return 0;
}
```

Output

Change the value of "ch" variable and check the output. For ch = 'm', we get the following output −

```
Time code: m

Good Morning
```

The use of **break** is very important here. The block of statements corresponding to each case ends with a **break** statement. What if the **break** statement is not used?

The switch-case works like this: As the program enters the switch construct, it starts comparing the value of switching expression with the cases, and executes the block of its first match. The **break** causes the control to go out of the switch scope. If **break** is not found, the subsequent block also gets executed, leading to incorrect result.

Example 2: Switch Statement without using Break

Let us comment out all the **break** statements in the above code.

</> Open Compiler

```
#include <stdio.h>

int main (){

   /* local variable definition */
   char ch = 'a';
   printf("Time code: %c\n\n", ch);

   switch (ch){
      case 'a':
         printf("Good Afternoon\n");
         // break;

      case 'e':
         printf("Good Evening\n");
```

```
         // break;

      case 'm':
         printf("Good Morning\n");
   }
   return 0;
}
```

Output

You expect the "Good Morning" message to be printed, but you find all the three messages printed!

```
Time code: a

Good Afternoon
Good Evening
Good Morning
```

This is because C falls through the subsequent case blocks in the absence of **break** statements at the end of the blocks.

Example 3: Grade Checker Program using Switch Statement

In the following program, "grade" is the switching variable. For different cases of grades, the corresponding result messages will be printed.

</> Open Compiler

```
#include <stdio.h>

int main(){

   /* local variable definition */
   char grade = 'B';

   switch(grade){
      case 'A' :
         printf("Outstanding!\n" );
         break;
      case 'B':
```

```
         printf("Excellent!\n");
         break;
      case 'C':
         printf("Well Done\n" );
         break;
      case 'D':
         printf("You passed\n" );
         break;
      case 'F':
         printf("Better try again\n" );
         break;
      default :
         printf("Invalid grade\n" );
   }
   printf("Your grade is  %c\n", grade);

   return 0;
}
```

Output

Run the code and check its output –

```
Excellent!
Your grade is  B
```

Now change the value of "grade" (it is a "char" variable) and check the outputs.

Example 4: Menu-based Calculator for Arithmetic Operations using Switch

The following example displays a menu for arithmetic operations. Based on the value of the operator code (1, 2, 3, or 4), addition, subtraction, multiplication or division of two values is done. If the operation code is something else, the default case is executed.

</> Open Compiler

```
#include <stdio.h>

int main (){
```

```
   // local variable definition
   int a = 10, b = 5;

   // Run the program with other values 2, 3, 4, 5
   int op = 1;
   float result;

   printf("1: addition\n");
   printf("2: subtraction\n");
   printf("3: multiplication\n");
   printf("4: division\n");

   printf("\na: %d b: %d : op: %d\n", a, b, op);
   switch (op){
      case 1:
         result = a + b;
         break;
      case 2:
         result = a - b;
         break;
      case 3:
         result = a * b;
         break;
      case 4:
         result = a / b;
         break;
      default:
         printf("Invalid operation\n");
   }
   if (op >= 1 && op <= 4)
      printf("Result: %6.2f", result);

   return 0;
}
```

Output

```
1: addition
2: subtraction
3: multiplication
4: division
```

```
a: 10 b: 5 : op: 1
Result:  15.00
```

For other values of "op" (2, 3, and 4), you will get the following outputs –

```
a: 10 b: 5 : op: 2
Result:   5.00

a: 10 b: 5 : op: 3
Result:  50.00

a: 10 b: 5 : op: 4
Result:   2.00

a: 10 b: 5 : op: 5
Invalid operation
```

Switch Statement by Combining Multiple Cases

Multiple cases can be written together to execute one block of code. When any of them case value matches, the body of these combined cases executes. If you have a situation where the same code block is to be executed for more than one case labels of an expression, you can combine them by putting the two cases one below the other, as shown below –

Syntax

```
switch (exp) {
   case 1:
   case 2:
      statements;
      break;
   case 3:
      statements;
      break;
   default:
      printf("%c is a non-alphanumeric character\n", ch);
}
```

You can also use the ellipsis (…) to combine a range of values for an expression. For example, to match the value of switching variable with any number between 1 to 10, you can use "case 1 … 10"

Example 1

</> Open Compiler

```
#include <stdio.h>

int main (){

   // local variable definition
   int number = 5;

   switch (number){

      case 1 ... 10:
         printf("The number is between 1 and 10\n");
         break;

      default:
         printf("The number is not between 1 and 10\n");
   }
   return 0;
}
```

Output

Run the code and check its output. For "number = 5", we get the following output –

```
The number is between 1 and 10
```

Example 2

The following program checks whether the value of the given char variable stores a lowercase alphabet, an uppercase alphabet, a digit, or any other key.

</> Open Compiler

```c
#include <stdio.h>

int main (){

   char ch = 'g';

   switch (ch){
      case 'a' ... 'z':
         printf("%c is a lowercase alphabet\n", ch);
         break;

      case 'A' ... 'Z':
         printf("%c is an uppercase alphabet\n", ch);
         break;

      case 48 ... 57:
         printf("%c is a digit\n", ch);
         break;

      default:
         printf("%c is a non-alphanumeric character\n", ch);
   }
   return 0;
}
```

```
g is a lowercase alphabet
```

Output

For **ch = 'g'**, we get the following output −

```
g is a lowercase alphabet
```

Test the code output with different values of "ch".